

Denoising-Resilient Latent Cryptographic Soft Bindings:

Architectures, Vulnerabilities, and Cross-Layer Provenance Integrity

Source guide

This research explores the intersection of latent representation spaces and digital security, detailing how high-dimensional neural data can be bound to cryptographic keys and content provenance metadata. The text identifies a critical distinction between "latent debt" in legacy software and the "latent manifolds" of machine learning, where soft bindings use perceptual hashes and watermarks to maintain an asset's identity even after its metadata is stripped. A significant portion of the work evaluates the vulnerabilities of these systems, such as denoising attacks and model inversion, while proposing advanced mathematical primitives like L2FE-Hash to protect against reconstruction threats. Ultimately, the source advocates for cross-layer audit protocols to resolve the "Integrity Clash," a state where valid digital signatures might contradict embedded watermarks regarding an asset's AI-generated origin.

Conceptual Dichotomy of Latent Cryptography: Software Debt vs. Neural Spaces

The terminology of "latent cryptography" has evolved along two separate paths in contemporary security engineering, reflecting different operational definitions of "latent" structures. In the domain of software supply chain security, the term refers to latent cryptographic debt.[1] This form of security debt represents hidden risks buried deep within software dependencies, legacy application code, and production infrastructure.[1, 2] Traditional vulnerability databases and Common Vulnerabilities and Exposures scanners often fail to flag these risks because they occur in components that are technically fully up-to-date.[1]

This latent risk is characterized by the silent persistence of deprecated algorithms, insufficient cryptographic key lengths, and non-compliant digital certificates.[1, 2] For example, automated discovery scans routinely surface active deployments of legacy internet key exchange protocols, weak Diffie-Hellman groups, outdated cipher suites like Triple DES, or the usage of outdated signature algorithms such as SHA-1.[1, 2] In these software environments, the term "latent" denotes an unquantified accumulation of technical debt that exposes organizations to future cryptanalysis, signature forgery, and post-quantum vulnerability.[2]

Conversely, in the domains of machine learning, biometric template protection, and content authenticity, "latent" refers to the high-dimensional mathematical manifold—the latent representation space—generated by deep neural networks.[3, 4, 5] To understand how the mathematical concept of latent representations has transitioned to cryptography, it is helpful to look at its origins in text mining and information retrieval.[6, 7] Historical modeling techniques such as Latent Semantic Indexing, Latent Semantic Analysis, Probabilistic Latent Semantic Analysis, and Latent Dirichlet Allocation mapped continuous co-occurrence patterns of text into lower-dimensional thematic spaces using Singular Value Decomposition or Bayesian Dirichlet priors.[6, 7]

While these early models uncovered hidden semantic topics in textual corpora, they struggled with short text structures due to severe sparsity, which required the development of specialized models like the Dirichlet Multinomial Mixture model.[6, 7]

In modern neural security systems, the latent space is used as a denoised, information-rich manifold where continuous, high-dimensional inputs are projected into highly stable vector clusters.[4, 5, 8] Deep Siamese Neural Networks and variational autoencoders compress noisy, real-world signals—such as physical side-channel power traces, biometric patterns, or media structures—into compact latent representations.[4, 5, 8]

Within this latent space, deep neural networks filter out random noise, isolating the core information manifold.[8] Latent cryptographic soft bindings operate directly on this denoised neural manifold, bridging continuous-valued neural representations with discrete, exact cryptographic keys and tamper-evident signatures.[3, 4, 9]

Technical Architecture of C2PA Soft Bindings and Manifest Recovery

To address the vulnerability of traditional metadata formats (such as EXIF or IPTC) to metadata stripping and unauthenticated editing, the Coalition for Content Provenance and Authenticity established an open cryptographic standard.[10, 11, 12] Under the C2PA specification, an asset's lineage is recorded within a signed metadata structure known as a C2PA manifest or Content Credential.[9, 10, 11]

A standard manifest is packaged within a JPEG Universal Metadata Box Format container and embedded directly inside the media file, or distributed externally as a sidecar file.[10, 11] The internal architecture of a C2PA manifest consists of three required components [10]:

- **Assertions:** Individual statements detailing the asset's origin, editing operations, and the involvement of generative AI models, often serialized as CBOR for compactness.[10, 13]
- **Claim:** A single, consolidated data structure that bundles all assertion hashes and references into a tamper-evident digest.[9, 10, 11]
- **Claim Signature:** A cryptographic signature computed over the claim using a trusted X.509 certificate and formatted using CBOR Object Signing and Encryption.[9, 10, 11]

To bind this declarative metadata structure to the actual media payload, the standard specifies two forms of binding.[9, 12, 14] A hard binding computes an exact cryptographic hash (typically SHA-256) over the defined byte ranges of the media, ensuring that any subsequent pixel-level alteration breaks the hash and invalidates the manifest.[9, 10, 11]

However, because routine web distribution and social media platforms regularly re-encode, scale, or compress files, hard bindings are highly fragile in practice.[3, 15, 16] This push toward real-world survivability drove the adoption of soft bindings, which leverage perceptual hashes or invisible watermarks to maintain an association between the asset's physical pixels and its signed provenance records.[3, 9, 16]

When an asset undergoes metadata stripping, the soft binding acts as a key to look up and recover the decoupled C2PA manifest from an online Manifest Repository.[17, 18] This paradigm is known as Durable Content Credentials, which unify metadata, invisible watermarks (such as Adobe's TrustMark), and passive image fingerprinting.[11, 19, 20]

Under this framework, if an asset is stripped of its metadata, the browser or validating client extracts a persistent watermark identifier from the pixels, queries the repository, and retrieves the signed manifest.[19, 21] To prevent watermark transfer or spoofing attacks—where an adversary copies a legitimate watermark onto fraudulent content—the retrieved manifest contains a signed, passive fingerprint of the original image.[19, 21] The validator computes a matching fingerprint of the queried asset, ensuring both the watermark and visual content align.[19, 21]

To facilitate global interoperability, C2PA manages a Soft Binding Algorithm List, which serves as an authoritative, JSON-structured registry of approved watermarking and fingerprinting technologies.[17, 22] This registry includes proprietary and open-source implementations, such as Adobe’s Image Comparator Network Dense Fingerprint, registered under the identifier com.adobe.icn.dense, and TrustMark Variant P, registered under com.adobe.trustmark.P.[18] Clients communicate with manifest databases using the C2PA Soft Binding Resolution API, which maps queries across four standardized route groups.[17]

Route Group	HTTP Method & Endpoint	Core Functional Purpose	Operational Scenario
Query	GET /matches/byBinding POST /matches/byBinding	Resolves matching manifest identifiers from a database given a known soft binding value and its corresponding algorithm ID.[17]	Preferred for rapid, URL-compatible queries or large payload retrievals.[17]
Query	POST /matches/byContent	Directly accepts a raw uploaded media asset, extracting or computing the soft binding on the server side to locate the manifest.[17]	Used when the validating client lacks the computational resource to extract the watermark.[17]

Query	POST /matches/byReference	Accepts a reference HTTPS URL pointing to a remote asset, instructing the repository to fetch the file and extract the binding.[17]	Optimizes bandwidth when dealing with massive files, such as long-form streaming video.[10, 17]
Store	POST /manifests	Ingests a new C2PA manifest store into the repository database, returning the active manifest ID and an ingestion receipt.[17]	Triggered by a creator's signing tool during export or cloud-based asset generation.[11, 17]
Store	POST /bindings PUT /bindings	Registers or updates a permanent association between a specific soft binding value and an active, stored manifest.[17]	Pairs a freshly embedded watermark identifier with its corresponding cryptographically signed manifest.[17]
Fetch	GET /manifests/{manifestId}	Retrieves a stored CBOR manifest store using its unique identifier, returning either the full historical store or only the active manifest.[10, 17]	The final resolution step in restoring the user-facing Content Credentials badge.[11, 17]

Service	GET /services/supportedAlgorithms	Returns the complete list of watermarking and fingerprinting algorithms supported by the manifest repository.[17]	Executed during initial client-server handshakes to negotiate compatible extraction models.[17]
---------	--------------------------------------	---	---

Latent-Space Watermarking and Generative Model Integration

While post-hoc watermarking techniques embed payload bits into synthesized assets as a distinct post-processing step via learned encoder-decoder networks, modern security models increasingly deploy latent-space watermarking.[3, 15, 16] Post-hoc architectures, such as Meta's Pixel Seal, utilize specialized networks (like MUNIT-based encoders with skip connections and post-processing 1×1 convolutions) to preserve high-fidelity visual structures while appending low-resolution residuals to the cover image.[15, 23, 24]

These models are trained end-to-end against a series of differentiable geometric, compression, and noise perturbations to ensure the decoder can recover the embedded watermark.[15, 25, 26]

In contrast, latent-space methods integrate the watermark directly into the latent representations during the generative sampling process of Latent Diffusion Models.[3, 16] Diffusion models synthesize media by iteratively denoising a latent tensor over a sequence of steps, starting from a random distribution.[25]

Latent-space watermarking algorithms, such as Tree-Ring and GaussianShading, embed their payload by applying mathematical constraints directly to this initial latent noise vector or by biasing the intermediate trajectory of the reverse diffusion steps.[3, 16] Because the watermarking signal is integrated into the underlying mathematical representation of the asset prior to spatial decoding, it becomes structurally part of the visual or auditory geometry.[3, 16] This makes the watermark extremely robust against post-hoc modifications without causing perceptible artifacts.[3, 15, 16]

For enterprise-scale tracking and complex asset lifecycles, soft bindings must also support fine-grained, multi-regional localization.[27] The NRGMark framework achieves this by dynamically partitioning an image into distinct, mixed-presence watermarked regions of arbitrary size.[27]

During training, these regions are assembled into composite training images.[27] To detect and localize these regions under distortion, NRGMark feeds the noisy asset into a localizer module

featuring a Region Proposal Network.[27] The network optimization is regulated via a multi-task loss function combining classification and regression targets [27]:

$$L_{rpn}$$

$$= \frac{1}{N_{cls}}$$

$$\sum_i$$

$$L_{cls}$$

$$(p_i - p_{gt_i})^2$$

$$)+\lambda$$

$$\frac{1}{N_{reg}}$$

$$\sum_i$$

$$p_i - p_{gt_i}$$

$$L_{reg}$$

$$(t_i - t_{pred_i})^2$$

$$L_{reg}$$

$$(t_i - t_{pred_i})^2$$

$$, t_{pred_i}$$

i
 $*$

)
In this formulation, N
 cls

and N
 reg

serve as normalization factors, while λ represents a balancing hyperparameter.[27] The
variable p
 i

is the predicted probability of anchor i being a watermarked region, p
 i
 $*$

is the ground-truth binary label, t
 i

represents the predicted parameterized bounding-box coordinates, and t
 i
 $*$

is the ground-truth target coordinate vector.[27]
Once localized, each predicted watermarked patch is extracted and passed to a dedicated
decoder to extract its independent binary secret.[27] This multi-regional capability enables
granular, tamper-resistant tracing of composite media assets.[23, 27]

Mathematical Modeling of Fuzzy Extractors and Key Binding in Latent Spaces

Generating stable cryptographic keys from continuous latent representation spaces requires specialized mathematical primitives.[4, 28] Because machine learning models map noisy physical measurements or biometric traits to continuous embedding vectors, they cannot be directly hashed using standard cryptographic functions.[4, 28] A standard hash function, like SHA-256, is designed to be highly sensitive; even a single-bit change in the input vector yields a completely different output.[28, 29]

To bridge this gap, biometric cryptosystems and property-preserving hashing frameworks deploy Fuzzy Extractors and Secure Sketches.[4, 29, 30] These cryptographic tools tolerate bounded input noise, enabling the reproduction of a deterministic, high-entropy secret key from varying inputs.[4, 29]

In the Adaptive Multimodal Biometric Key Generation framework, a Deep Siamese Neural Network is trained to project biometric traits (face, iris, and fingerprint) into a highly stable, tightly clustered latent space.[4, 5] To generate a cryptographic key, the continuous latent vector V fused

is quantized into a stable n -bit binary string $w \in \{0,1\}^n$

[4]

During the enrollment phase (Gen), a secret seed $c \in \{0,1\}^k$

k

is randomly sampled from a Bose-Chaudhuri-Hocquenghem error-correcting code, defined by the parameters [4]:

$BCH(n,k,t)$

where t represents the maximum correctable bit-error threshold.[4] The system then generates a public helper data structure (the Secure Sketch) P by performing a bitwise XOR operation between the quantized binary vector w and the encoded representation of the seed [4]:

$P = w \oplus ECCEncode(c)$

During authentication, a fresh, noisy query vector is quantized to produce w' ,

[4] Using the public helper data P , the reconstruction phase (Rep) attempts to recover the original secret seed c [4]:

$c = BCHDecode(w'$

,

$\oplus P)$

This reconstruction succeeds if and only if the Hamming distance between the enrollment string and the query string is within the correctable limit [4]:

d

H

(w, w')

,

$) \leq t$

Once recovered, the seed is processed through a key derivation function to generate a stable cryptographic key.[4, 30]

- Continuous Feature Vector (V_{fused}) $\rightarrow$ [Quantizer] $\rightarrow$ Binary String (w)
- Secret Seed (c) $\rightarrow$ (XOR) $\rightarrow$ Helper Data (P)

In property-preserving hashing, a perceptual binary code $c(x)$ is extracted from a ResNet-18 model via linear projection and sign quantization.[30] This code serves as the noisy source for a fuzzy extractor to generate a reproducible secret R and public data P , which are then processed using a key derivation function and HMAC/SHA-3 to produce a secure cryptographic tag.[30]

However, because many deep learning applications natively compare vectors using Euclidean

(L

2

) distance rather than Hamming distance, traditional fuzzy extractors can be inefficient or insecure.[28, 29] To address this limitation, security researchers formalized the L2FE-Hash primitive, a fuzzy extractor designed natively for the L

2

norm.[28, 29, 31]

An ideal Euclidean post-processing primitive must satisfy three properties [29]:

- Correctness (Noise Tolerance): For any enrolled vector x and subsequent query x' ,
- if $\|x - x'\|_2 \leq \theta$, then the reproduction algorithm succeeds, ensuring $\text{Rep}(x, P) = y$ with high probability.[29]
- Security (Fuzzy One-Wayness): Given the public helper data P and the target hash value y , it is computationally infeasible for an adversary to reconstruct any vector x' that is close to the original enrolled vector x . [29]
- Utility: The entropy of the generated output y , conditioned on the public helper data P , must be comparable to the entropy of the original input space, preserving downstream matching accuracy without requiring model retraining.[28, 29]

Vulnerability Analysis: Denoising Attacks, Model Inversion, and the PIPE Threat

Integrating cryptographic bindings into continuous latent representations introduces several unique security threats.[3, 26, 28] Because soft bindings and watermarks are typically embedded in the high-frequency bands of an asset, they are highly sensitive to denoising operations.[26]

Adversaries exploit this vulnerability using specialized watermark removal networks, such as ReMark, which treat watermark stripping as an image restoration and denoising task.[26]

ReMark utilizes deep restoration backbones like KBNNet and is trained using a loss function that balances structural fidelity against watermark decoder failure [26]:

L

ReMark

$= \alpha L$

MSE

$+ \beta L$

GAN+GP

+ γL

X

where L

MSE

prioritizes peak signal-to-noise ratio, L

GAN+GP

ensures realistic textures, and L

X

represents the watermark similarity loss, which the optimization minimizes to eliminate the embedded secret.[26]

Conversely, "denoising-resilient" cryptographic techniques are designed to resist standard noise-reduction filters.[32] In voice obfuscation architectures like SecHeadset, phoneme-based, voice-like noise masking is designed to protect voice privacy during transmissions over unsecure communication channels.[32]

Because this noise mimics the acoustic and phonemic characteristics of human speech, standard noise suppression filters (used by adversaries or lossy VoIP channels) mistakenly preserve the mask.[32] This ensures strong denoising resilience and secures information exchange through real-time channel state estimation protocols.[32]

In physical side-channel cryptographic attacks, Supervised Variational Autoencoders are used to map high-dimensional, noisy power traces (compressing a 700-dimensional raw trace down to a 32-dimensional latent representation) to expose latent leaks.[8] This allows conditional Denoising Diffusion Implicit Models to synthesize side-channel traces, enabling profiled attacks against post-quantum implementation platforms, such as Artix-7 FPGA boards running CRYSTALS-Kyber.[8, 33]

Beyond physical watermark stripping and side-channel analysis, latent-space systems face severe privacy threats from embedding inversion and model inversion attacks.[28, 34] In these scenarios, an attacker gains access to a database of stored high-dimensional embedding vectors.[28, 29]

Using these leaked vectors, the adversary trains a surrogate model to map the latent representations back to the raw source inputs.[28, 34] This enables highly accurate reconstructions of sensitive biometric templates or personal identities.[28, 29]

The state-of-the-art model inversion threat is exemplified by the PIPE attack.[28, 29, 35] Prior post-processing defenses, such as random projection and early-generation facial fuzzy extractors, attempted to protect continuous embeddings by adding noise or projecting vectors into randomized lower-dimensional spaces.[28, 29]

The PIPE attack bypasses these defenses by training surrogate models to map protected representations back to raw inputs, achieving reconstruction success rates exceeding 89%

across legacy security frameworks.[28, 29, 35] L2FE-Hash addresses this vulnerability by enforcing a cryptographic one-way barrier directly over the L

2

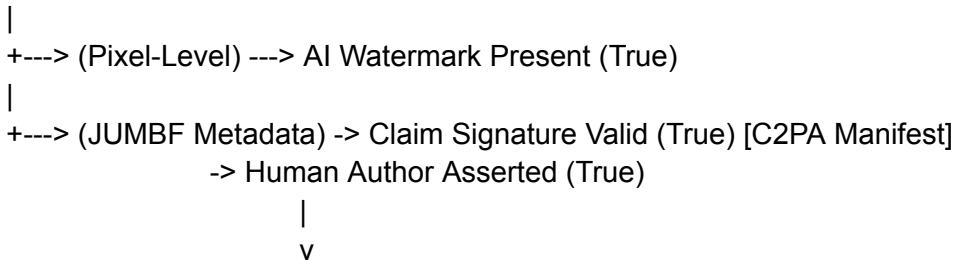
norm, reducing PIPE reconstruction success rates to baseline levels while preserving legitimate verification accuracy.[28, 29]

Cross-Layer Desynchronization: The Integrity Clash and Forensic Auditing

A structural vulnerability in modern digital provenance architectures is the lack of coordination between metadata provenance standards (C2PA) and pixel-level watermarking layers.[3, 16, 36] Because these verification systems operate independently, their semantic assertions can diverge.[3, 15, 36] This architectural gap allows a user to pass an AI-generated, watermarked asset through a C2PA-compliant editor, sign it with a valid certificate, and omit the generative origin assertion entirely.[3, 15, 36]

The resulting asset carries a cryptographically valid C2PA manifest claiming exclusive human authorship, while its physical pixels simultaneously contain a watermarking signal (such as Google's SynthID or Meta's Pixel Seal) identifying it as AI-generated.[3, 15, 36] Both layers verify successfully in isolation, producing a contradictory authentication state.[3, 15, 36]

[AI-Generated Asset]



To categorize these cross-layer desynchronization states, researchers developed a cross-layer conflict matrix that maps the interaction between C2PA manifests (M) and physical watermarks (W).[15, 37]

Conflict Matrix Quadrant	Manifest Status (M)	Watermark Status (W)	Core Semantic Interpretation	System Vulnerability & Forensic Implications

Q1: Silent Zone	Absent or Invalid [3, 15]	Not Detected [3, 15]	The asset contains no machine-readable authenticity signals.[3, 15]	Standard state for un-credentialed media; offers no tracking capability.[3, 15]
Q2: Fragile Provenance	Absent or Invalid [15]	Detected (AI Mark) [15]	The physical pixels carry an AI watermark, but no metadata exists.[15]	High probability that the original C2PA metadata was stripped.[15, 24]
Q3: Authenticated Content	Valid (Human Claim) [15]	Not Detected [15]	Cryptographic metadata asserts human origin, with no contradictory pixel watermark.[15]	Represents the standard human-authored asset lifecycle.[15]
Q4a: Verified Synthetic	Valid (AI Disclosed) [37]	Detected (AI Mark) [37]	Both the manifest and the physical pixels consistently declare AI origin.[37]	Robust alignment; represents the intended behavior of the metadata standard.[37]
Q4b: Authenticated Fake	Valid (Human Claim) [15, 37]	Detected (AI Mark) [15, 37]	The manifest claims human authorship, but the pixels contain an AI watermark.[15, 36, 37]	Integrity Clash: Highlights the limits of declarative metadata.[3, 15, 36]

Resolving the Q4b Integrity Clash requires moving away from isolated single-layer verification.[36, 37] To close this gap, security architectures propose Cross-Layer Audit Protocols that jointly evaluate declarative metadata claims against physical pixel watermarks.[16, 36, 37]

By enforcing logical consistency rules—such as requiring any asset containing an active synthetic watermark to include a corresponding compositeWithTrainedAlgorithmicMedia assertion in its active C2PA manifest—these protocols successfully detect semantic contradictions.[10, 37] In empirical evaluations across thousands of perturbed test assets, these joint audit systems achieved 100% accuracy in identifying desynchronized metadata claims, demonstrating a practical path forward for securing digital media ecosystems.[37, 38]

Strategic Conclusions

Securing digital content provenance requires addressing several structural vulnerabilities across declarative metadata, physical watermarks, and continuous latent spaces.[3, 29, 36] Declarative standards like C2PA provide robust tamper-evidence for metadata, but they ultimately rely on the honesty of the signer.[3, 11, 36] Because metadata can be modified or stripped during distribution, systems must deploy soft bindings (such as Adobe’s TrustMark or Meta’s Pixel Seal) to enable manifest recovery and cross-checking.[3, 9, 19] However, the independent design of these verification layers makes them vulnerable to the Integrity Clash, where valid signatures can authenticate false claims.[3, 36, 37]

To prevent these contradictions, verification platforms must transition to Cross-Layer Audit Protocols that jointly evaluate both metadata assertions and physical pixel states.[36, 37, 38] Furthermore, as watermarking and key extraction move into deep latent spaces, systems must protect these high-dimensional manifolds against adversarial denoising and model inversion.[26, 28, 29]

Traditional post-processing defenses are vulnerable to advanced attacks like PIPE.[28, 29] This requires deploying L

2

-optimized cryptographic primitives, such as L2FE-Hash, to secure latent embeddings and maintain end-to-end provenance integrity.[28, 29]

-
1. Explore Cryptographic Inventory and Risk in Open-Source Software | AQtive Guard, <https://opencryptography.com/about>
 2. Crypto Discovery Findings Across Enterprise Environments | QuantumGate, <https://quantumgate.ae/what-we-find-when-we-run-a-crypto-discovery-common-findings-across-environments>
 3. Authenticated Contradictions from Desynchronized Provenance and Watermarking - arXiv, <https://arxiv.org/html/2603.02378v2>
 4. Adaptive Multimodal Biometric Key Generation: A Deep ... - RJPN, <https://rjpn.org/ijnti/papers/IJNTI2604167.pdf>

5. Improving Biometric Security Through Multimodal Fusion and Deep Hashing - International Research Journal on Advanced Science Hub,
<https://rspsciencehub.com/index.php/journal/article/download/1027/872/1776>
6. A Detailed Survey on Topic Modeling for Document and Short Text Data,
<https://www.ijcaonline.org/archives/volume178/number39/likhitha-2019-ijca-919265.pdf>
7. A Detailed Survey on Topic Modeling for Document and Short Text Data - ResearchGate,
https://www.researchgate.net/publication/335339697_A_Detailed_Survey_on_Topic_Modeling_for_Document_and_Short_Text_Data
8. Less is More: Latent Diffusion for Efficient IoT Side-Channel Analysis,
<https://cetinkayakoc.net/docs/j103.pdf>
9. C2PA Glossary: Every Content Provenance Term Explained, <https://c2pa.ai/glossary>
10. What is a C2PA Manifest? Structure, Assertions, and Verification,
<https://c2paviewer.com/articles/what-is-c2pa-manifest>
11. How C2PA Content Credentials Work and What Their Limits Are - SoftwareSeni,
<https://www.softwareseni.com/how-c2pa-content-credentials-work-and-what-their-limits-are/>
12. Multimedia content authenticity and provenance - Fluendo,
<https://fluendo.com/blog/multimedia-content-authenticity-and-provenance/>
13. C2PA and Content Credentials Explainer,
<https://spec.c2pa.org/specifications/specifications/2.4/explainer/Explainer.html>
14. A DeepMark's Guide to C2PA: From Manifests to Soft-Bindings,
<https://www.deepmark.me/blog/a-deepmarks-guide-to-c2pa-from-manifests-to-soft-bindings>
15. Authenticated Contradictions from Desynchronized Provenance and Watermarking - arXiv, <https://arxiv.org/html/2603.02378v1>
16. Authenticated Contradictions from Desynchronized Provenance and Watermarking - NSF Public Access Repository, <https://par.nsf.gov/servlets/purl/10684077>
17. C2PA Soft Binding API :: C2PA Specifications,
<https://spec.c2pa.org/specifications/specifications/2.4/softbinding/Decoupled.html>
18. CAI Soft binding resolution API - Adobe Developer,
<https://developer.adobe.com/cai-soft-binding-api/>
19. Three pillars of provenance that make up durable Content Credentials,
<https://contentauthenticity.org/blog/three-pillars-of-provenance>
20. Durable Content Credentials | Open-source tools for content authenticity and provenance, <https://opensource.contentauthenticity.org/docs/durable-cr/>
21. From Detection to Provenance: Securing Media Authenticity with Content Credentials,
<https://crestresearch.ac.uk/comment/from-detection-to-provenance-securing-media-authenticity-with-content-credentials/>
22. A list of C2PA approved soft binding algorithms - GitHub,
<https://github.com/c2pa-org/softbinding-algorithm-list>
23. InvisMark: Invisible and Robust Watermarking for AI-Generated Image Provenance - CVF Open Access,

- https://openaccess.thecvf.com/content/WACV2025/papers/Xu_InvisMark_Invisible_and_Robust_Watermarking_for_AI-Generated_Image_Provenance_WACV_2025_paper.pdf
24. InvisMark: Invisible and Robust Watermarking for AI-generated Image Provenance - arXiv, <https://arxiv.org/html/2411.07795v1>
 25. An early look at cryptographic watermarks for AI-generated content - The Cloudflare Blog, <https://blog.cloudflare.com/an-early-look-at-cryptographic-watermarks-for-ai-generated-content/>
 26. TrustMark: Robust Watermarking and Watermark Removal for Arbitrary Resolution Images - CVF Open Access, https://openaccess.thecvf.com/content/ICCV2025/papers/Bui_TrustMark_Robust_Watermarking_and_Watermark_Removal_for_Arbitrary_Resolution_Images_ICCV_2025_paper.pdf
 27. NRGMark: Localized Watermarking for Energy Transparency in Images - Prof. John Collomosse, <https://collomosse.com/pubs/Agarwal-WACV-2026.pdf>
 28. Model Inversion Attacks Meet Cryptographic Fuzzy Extractors - ResearchGate, https://www.researchgate.net/publication/397041270_Model_Inversion_Attacks_Meet_Cryptographic_Fuzzy_Extractors
 29. Model Inversion Attacks Meet Cryptographic Fuzzy Extractors - arXiv, <https://arxiv.org/html/2510.25687v1>
 30. Learning to Break Deep Perceptual Hashing: The Use Case NeuralHash - ResearchGate, https://www.researchgate.net/publication/361430168_Learning_to_Break_Deep_Perceptual_Hashing_The_Use_Case_NeuralHash
 31. Cryptographic Primitives for ML Security - MAPS, <https://maps.kisp-lab.org/problems/crypto-mlsec>
 32. SecHeadset: A Practical Privacy Protection System for Real-time Voice Communication, <https://desperado1999.github.io/assets/pdf/secheadset.pdf>
 33. Denoising of Power Traces in Side-Channel Analysis - DiVA portal, <https://www.diva-portal.org/smash/get/diva2:1887939/FULLTEXT01.pdf>
 34. Embedding Inversion Attacks - Emergent Mind, <https://www.emergentmind.com/topics/embedding-inversion-attacks>
 35. [2510.25687] Model Inversion Attacks Meet Cryptographic Fuzzy Extractors - arXiv, <https://arxiv.org/abs/2510.25687>
 36. Authenticated Contradictions from Desynchronized Provenance and Watermarking - CVF Open Access, https://openaccess.thecvf.com/content/CVPR2026W/APAI/papers/Nemecek_Authenticated_Contradictions_from_Desynchronized_Provenance_and_Watermarking_CVPRW_2026_paper.pdf
 37. (PDF) Authenticated Contradictions from Desynchronized Provenance and Watermarking, https://www.researchgate.net/publication/401523975_Authenticated_Contradictions_from_Desynchronized_Provenance_and_Watermarking

38. [2603.02378] Authenticated Contradictions from Desynchronized Provenance and Watermarking - arXiv, <https://arxiv.org/abs/2603.02378>